



TUGAS AKHIR - ET234801

Nihilisme dan Eksistensialisme Pada *Anime* Monster

Elon Reeve Musk

NRP 0123 20 4000 0001

Dosen Pembimbing

Nikola Tesla, S.T., M.T

NIP 18560710 194301 1 001

Wernher von Braun, S.T., M.T

NIP 18560710 194301 1 001

Program Studi Strata 1 (S1) Teknologi Informasi

Departemen Teknologi Informasi

Fakultas Teknik Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

LEMBAR PENGESAHAN

Nihilisme dan Eksistensialisme Pada *Anime* Monster

PROPOSAL TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Teknik pada
Program Studi S-1 Teknologi Informasi
Departemen Teknologi Informasi
Fakultas Teknik Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh: **Elon Reeve Musk**
NRP. 0123 20 4000 0001

Disetujui oleh Tim Penguji Tugas Akhir:

Nikola Tesla, S.T., M.T
NIP: 18560710 194301 1 001

(Pembimbing)

Wernher von Braun, S.T., M.T
NIP: 18560710 194301 1 001

(Ko-pembimbing)

SURABAYA
Juni, 2026

APPROVAL SHEET

Nihilism and Existentialism in Monster Anime

FINAL PROJECT PROPOSAL

Submitted to fulfill one of the requirements
for obtaining a degree Bachelor of Engineering at
Undergraduate Study Program of Information Technology
Department of Information Technology
Faculty of Intelligent Electrical and Informatics Technology
Sepuluh Nopember Institute of Technology

By: **Elon Reeve Musk**
NRP. 0123 20 4000 0001

Approved by Final Project Examiner Team:

Nikola Tesla, S.T., M.T
NIP: 18560710 194301 1 001

(Advisor)

Wernher von Braun, S.T., M.T
NIP: 18560710 194301 1 001

(Co-Advisor)

SURABAYA
June, 2026

ABSTRAK

Nihilisme dan Eksistensialisme Pada *Anime* Monster

Nama Mahasiswa / NRP : Elon Reeve Musk / 0123 20 4000 0001
Department : Teknologi Informasi - Fakultas Teknik Elektro dan Informatika
Cerdas
Dosen Pembimbing : 1. Nikola Tesla, S.T., M.T
2. Wernher von Braun, S.T., M.T

Pada penelitian ini kami mengajukan Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Kata Kunci: Paranoid Android, Karma Police, Fake Plastic Trees, Just, High and Dry, No Surprises, Creep, Exit Music (For a Film), Let Down

ABSTRACT

Nihilism and Existentialism in Monster Anime

Name / NRP : Elon Reeve Musk / 0123 20 4000 0001
Department : Information Technology - Faculty of Intelligent Electrical and Informatics Technology
Advisor : 1. Nikola Tesla, S.T., M.T
2. Wernher von Braun, S.T., M.T

Pada penelitian ini kami mengajukan Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Keywords: Paranoid Android, Karma Police, Fake Plastic Trees, Just, High and Dry, No Surprises, Creep, Exit Music (For a Film), Let Down

DAFTAR ISI

ABSTRAK	i
ABSTRAK	ii
DAFTAR ISI	iii
DAFTAR GAMBAR	v
DAFTAR TABEL	vi
1 Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan	2
1.5 Manfaat	3
2 Tinjauan Pustaka	5
2.1 Hasil Penelitian Terdahulu	5
2.2 Roket Luar Angkasa	6
2.3 Gravitasi	7
2.3.1 Hukum Newton	7
2.3.2 Anti Gravitasi	7
3 Metodologi	9
3.1 Deskripsi Sistem	9
3.2 Implementasi Alat	9
3.3 Implementasi	9
3.3.1 Penyiapan Lingkungan Sistem	9
3.3.2 Ekstraksi Data Wajah	10
3.3.3 Sinkronisasi Identitas	11
3.3.4 Image Preprocessing	14
3.3.5 Enkripsi Embedding	19
	iii

3.3.6	Pelatihan Model	20
3.3.7	Evaluasi Sistem	24
3.3.8	Proses Absensi	28
3.3.9	Pengembangan Antarmuka Aplikasi	31

DAFTAR PUSTAKA	33
-----------------------	-----------

DAFTAR GAMBAR

2.1	Peluncuran roket luar angkasa <i>Discovery</i> (“Roket Luar Angkasa Discovery”, 2021).	6
3.1	Tampilan Antarmuka Dasbor Utama <i>Server</i>	31
3.2	Tampilan Antarmuka Terminal Presensi Sisi <i>Client</i>	32

DAFTAR TABEL

2.1	Penelitian Terdahulu	5
-----	--------------------------------	---

BAB I Pendahuluan

Penelitian tugas akhir ini membahas tentang penerapan *Federated Learning* pada sistem kehadiran berbasis pengenalan wajah di lingkungan *Edge Computing* untuk mengatasi isu privasi data biometrik dan efisiensi *bandwidth*.

1.1 Latar Belakang

Perkembangan kecerdasan buatan (AI) dalam beberapa tahun terakhir berlangsung sangat pesat dan memberikan dampak signifikan pada berbagai sektor seperti industri, pendidikan, kesehatan, dan transportasi. Kemampuan AI dalam mengolah data dalam jumlah besar serta menemukan pola tertentu membuat proses pengambilan keputusan menjadi lebih cepat dan akurat. Di saat yang sama, penggunaan perangkat *Internet of Things* (IoT) juga semakin meluas sehingga menghasilkan lebih banyak data dari perangkat yang saling terhubung. Namun, perkembangan AI dan IoT ini menghadirkan tantangan baru, terutama terkait keamanan data, privasi pengguna, dan kompleksitas dalam pengelolaannya (Brecko et al., 2022).

Sebagian besar sistem AI masih menggunakan metode *Centralized Learning*, di mana seluruh data dikumpulkan ke satu *server* pusat untuk dilatih. Pendekatan ini memang efektif, tetapi kurang ideal karena berpotensi menimbulkan kebocoran data pribadi serta memerlukan *bandwidth* yang besar, terutama ketika data mentah dikirim dari banyak perangkat IoT (Nandi et al., 2023). Permasalahan ini menjadi semakin menonjol pada aplikasi yang memanfaatkan data biometrik, seperti pengenalan wajah, karena jenis data tersebut sangat sensitif dan tidak dapat diganti jika terjadi kebocoran.

Pengenalan wajah merupakan salah satu teknologi yang banyak digunakan dalam sistem kehadiran karena mampu mengotomatiskan proses identifikasi dan mengurangi ketergantungan pada metode absensi manual. Namun, penggunaan citra wajah sebagai data biometrik menimbulkan persoalan penting terkait privasi, terutama ketika data perlu diproses atau disimpan di *server* pusat. Srinu et al. (2025) mengembangkan sistem kehadiran berbasis *convolutional neural network* (CNN) yang dijalankan pada perangkat *edge* seperti Raspberry Pi dan Jetson Nano, di mana proses inferensi dilakukan secara lokal untuk membatasi pergerakan data sensitif. Meskipun demikian, proses pelatihan model pada penelitian tersebut masih dilakukan secara terpusat sehingga data wajah tetap harus dikumpulkan ke *server* ketika pembaruan model diperlukan. Hal ini menunjukkan bahwa isu privasi pada tahap pelatihan belum sepenuhnya teratasi.

Untuk mengurangi ketergantungan pada pelatihan terpusat, *Federated Learning* diperkenalkan sebagai pendekatan yang memungkinkan pelatihan model dilakukan langsung di perangkat pengguna. Dalam *Federated Learning*, data mentah tidak perlu dikirim ke *server* pusat, hanya parameter atau pembaruan model yang dikirim untuk digabungkan menjadi model global (Staño et al., 2025). Dengan mekanisme tersebut, privasi data biometrik dapat lebih terjaga dan kebutuhan *bandwidth* menjadi lebih rendah. Penelitian Kim et al. (2021) juga menunjukkan bahwa *Federated Learning* efektif untuk melatih model pengenalan wajah karena citra wajah pengguna tetap berada di perangkat masing-masing. Meskipun temuan tersebut menunjukkan potensi besar *Federated Learning* untuk aplikasi biometrik, hingga saat ini belum terdapat penelitian yang menerapkan *Federated Learning* pada sistem kehadiran berbasis pengenalan wajah di perangkat

edge, sehingga menyisakan celah penelitian yang masih perlu dieksplorasi.

Berdasarkan permasalahan tersebut, penelitian tugas akhir ini mengusulkan penerapan *Federated Learning* pada sistem kehadiran berbasis pengenalan wajah di lingkungan *Edge Computing*. Dalam penelitian ini, model *MobileFaceNet* akan dilatih secara lokal di perangkat *client*, sehingga citra wajah tidak perlu dikirim ke *server* pusat. Proses distribusi dan orkestrasi *server* dan *client* akan menggunakan *Docker* agar sistem lebih mudah dijalankan dan direplikasi. Harapannya, kombinasi *Federated Learning*, *MobileFaceNet*, dan *Edge Computing* dapat menghasilkan sistem kehadiran yang lebih aman, efisien, serta tetap menjaga privasi pengguna.

1.2 Rumusan Masalah

Berdasarkan permasalahan tersebut, rumusan masalah yang diangkat pada penelitian adalah sebagai berikut:

1. Bagaimana membangun sistem kehadiran berbasis pengenalan wajah dan memanfaatkan konsep *Federated Learning* pada lingkungan *Edge Computing*?
2. Bagaimana membandingkan performa model *MobileFaceNet* antara *Federated Learning* dan *Centralized Learning*?

1.3 Batasan Masalah

Adapun batasan masalah dalam penelitian ini berdasarkan rumusan masalah adalah sebagai berikut:

1. Penelitian berfokus pada implementasi arsitektur *Federated Learning*, bukan pengembangan arsitektur model baru.
2. Model yang digunakan adalah *MobileFaceNet* sebagai *backbone* pengenalan wajah.
3. Sistem tidak mencakup fitur *anti-spoofing* atau deteksi keaslian wajah.
4. Lingkungan pengujian menggunakan sebuah Raspberry Pi dan Jetson Nano sebagai *perangkat edge* dan PC sebagai *server*.
5. Implementasi sistem berbasis *containerization* menggunakan *Docker*.
6. Objek citra wajah yang digunakan dalam eksperimen dibatasi pada maksimal 30 subjek mahasiswa Departemen Teknologi Informasi.
7. Proses akuisisi citra wajah dan pengujian sistem dilakukan menggunakan perangkat *webcam*.

1.4 Tujuan

Merujuk pada rumusan masalah, tujuan yang ingin dicapai:

1. Membangun sistem kehadiran berbasis pengenalan wajah dengan memanfaatkan konsep *Federated Learning* yang memungkinkan pelatihan model dilakukan secara lokal pada perangkat *edge*.
2. Membandingkan performa model *MobileFaceNet* yang dilatih menggunakan *Federated Learning* dengan *Centralized Learning* untuk menilai efektivitas *Federated Learning* dalam sistem kehadiran.

1.5 Manfaat

Manfaat yang diharapkan dari penelitian ini adalah sebagai berikut:

1. Mempermudah proses absensi dengan sistem pengenalan wajah.
2. Menjaga privasi pengguna karena citra wajah tidak dikirim ke *server* pusat.
3. Mengurangi kebutuhan *bandwidth* selama proses pelatihan model.

[Halaman ini sengaja dikosongkan]

BAB II Tinjauan Pustaka

Demi mendukung penelitian ini, Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque.

2.1 Hasil Penelitian Terdahulu

Beberapa penelitian terdahulu yang relevan dengan topik ini antara lain Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque.

Tabel 2.1: Penelitian Terdahulu

No	Penelitian	Pembahasan
1	Voyager (Wang et al., 2023)	Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Tabel 2.1 – *Lanjutan dari halaman sebelumnya*

No	Penelitian	Pembahasan
2	Model Context Protocol (Anthropic, 2024)	Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2.2 Roket Luar Angkasa



Gambar 2.1: Peluncuran roket luar angkasa *Discovery* (“Roket Luar Angkasa Discovery”, 2021).

Roket luar angkasa merupakan Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper

nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Discovery, Gambar 2.1, merupakan Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

2.3 Gravitasi

Gravitasi merupakan Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2.3.1 Hukum Newton

Newton (Newton, 1687) pernah merumuskan bahwa Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum. Kemudian menjadi persamaan seperti pada persamaan 2.1.

$$\sum \mathbf{F} = 0 \Leftrightarrow \frac{d\mathbf{v}}{dt} = 0. \quad (2.1)$$

2.3.2 Anti Gravitasi

Anti gravitasi merupakan Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus.

Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

BAB III Metodologi

3.1 Deskripsi Sistem

3.2 Implementasi Alat

3.3 Implementasi

Berikut merupakan tahapan implementasi pelaksanaan penelitian ini:

3.3.1 Penyiapan Lingkungan Sistem

Tahap awal pelaksanaan implementasi sistem dilakukan dengan mengonfigurasi seluruh lingkungan perangkat lunak, pustaka pemrograman, dan isolasi kontainerisasi pada sisi *server* maupun *client*. Langkah kontainerisasi ini krusial untuk menjamin kelancaran komunikasi data dan keselarasan dependensi sistem terdistribusi tanpa terikat oleh sistem operasi bawaan perangkat keras. Proses penyiapan lingkungan operasional ini dibagi menjadi dua poin utama sebagai berikut:

3.3.1.1 Client

Implementasi lingkungan pada sisi *client* (Jetson Nano dan Raspberry Pi) diisolasi menggunakan layanan kontainerisasi untuk mendukung sistem komputasi berbasis prosesor ARM yang tertanam pada perangkat keras *edge*. Sistem menggunakan *image* dasar resmi python:3.10-slim-bullseye karena varian ini sangat ringan, stabil, dan memiliki kecocokan arsitektur kompilasi silang yang optimal.

Di dalam kontainer ini, sistem mengonfigurasi beberapa parameter lingkungan penting, meliputi tautan alamat basis data SQLite lokal untuk menyimpan data kehadiran sementara, serta variabel alamat *server* utama dan alamat *server Federated Learning* untuk mengarahkan jalur komunikasi *socket* menuju alamat IP *server* pusat. Guna menjembatani interaksi visual secara langsung, kontainer diberikan hak akses khusus dan pemetaan perangkat fisik untuk mengaktifkan modul kamera *webcam* eksternal secara langsung, lengkap dengan pengaturan format video MJPG serta batas resolusi tangkapan berdimensi 1280x720 piksel.

3.3.1.2 Server

Implementasi lingkungan pada sisi *server* pusat dirancang secara multi-kontainer menggunakan berkas konfigurasi yang mengorkestrasi dua layanan utama, yaitu mesin aplikasi utama *Federated Learning* dan sistem manajemen basis data terpusat. Sama halnya dengan sisi *client*, kontainer aplikasi utama *server* dibangun menggunakan *image* dasar python:3.10-slim-bullseye guna menjaga efisiensi konsumsi memori RAM dan keselarasan interpretasi tipe data saat proses serialisasi parameter bobot berlangsung.

Server membuka dua jalur pintu gerbang komunikasi internal, yaitu gerbang pertama untuk melayani penarikan data REST API dari antarmuka *web* Flask, serta gerbang kedua untuk mengelola lalu lintas sinkronisasi parameter bobot lokal algoritma FedProx menggunakan pustaka Flower. Sementara itu, untuk menjamin keamanan rekapitulasi log absensi dari seluruh

client, *server* mengintegrasikan layanan basis data relasional berbasis *image* kontainer resmi postgres:15 yang dikonfigurasi dengan kredensial otentikasi yang aman.

3.3.2 Ekstraksi Data Wajah

Tahap pengelolaan data wajah awal merupakan fase pra-pelatihan yang dieksekusi secara mandiri di luar lingkungan operasional kontainer *server* maupun *client* menggunakan perangkat laptop. Proses ini dilakukan dengan membaca setiap bingkai dari berkas rekaman video mentah secara berurutan melalui fungsi pemrosesan citra dari pustaka OpenCV. Sistem kemudian mengambil nilai laju bingkai asli dari properti video tersebut untuk dikalikan dengan konstanta laju *sampling* sebesar 0,17 detik, sehingga diperoleh angka interval lompatan bingkai komputasi yang konstan. Setiap kali indeks bingkai berjalan mencapai kelipatan dari nilai interval tersebut, sistem akan mengeksekusi pemotongan dan penyimpanan citra statis secara terformat ke dalam direktori luaran. Proses penyimpanan ini dikonfigurasi dengan menyematkan tingkat kompresi kualitas JPEG yang tinggi sebesar 95% guna menjaga ketajaman resolusi objek wajah tanpa memperbesar ukuran berkas secara berlebihan. Alur implementasi ekstraksi visual ini ditunjukkan pada Kode Sumber 3.1.

Listing 3.1: Pemotongan Video Berdasarkan *Frame Rate*.

```
1 import os
2 import cv2
3
4 SAMPLING_RATE = 0.17
5
6 def run_extraction():
7     # Cek folder output
8     if not os.path.exists(OUTPUT_DIR):
9         os.makedirs(OUTPUT_DIR)
10
11     # Load video
12     cap = cv2.VideoCapture(VIDEO_SOURCE)
13     if not cap.isOpened():
14         return
15
16     # Ambil info FPS video asli
17     fps = cap.get(cv2.CAP_PROP_FPS)
18     interval = int(fps * SAMPLING_RATE)
19
20     frame_idx = 0
21     saved_idx = 0
22
23     while True:
24         ret, frame = cap.read()
25         if not ret:
26             break
27
28         # Simpan frame berdasarkan interval
29         if frame_idx % interval == 0:
30             filename = f"frame_{frame_idx:06d}.jpg"
31             save_path = os.path.join(OUTPUT_DIR, filename)
32             # Simpan gambar dengan kualitas JPEG 95%
33             cv2.imwrite(
34                 save_path, frame,
35                 [int(cv2.IMWRITE_JPEG_QUALITY), 95]
36             )
```

```
37         saved_idx += 1
38
39         frame_idx += 1
40
41     cap.release()
```

3.3.3 Sinkronisasi Identitas

Tahap penyelarasan identitas merupakan fase krusial dalam arsitektur pembelajaran terdistribusi yang bertujuan untuk menstandarkan indeks label kelas mahasiswa dari seluruh *client* secara terpusat sebelum pelatihan model dimulai. Langkah ini dirancang untuk mengeliminasi risiko terjadinya tumpang tindih penomoran target matriks klasifikasi jala saraf MobileFaceNet antarperangkat *edge* yang mengoleksi subset data berbeda. Mekanisme ini dieksekusi melalui empat tahapan sinkronisasi asinkron antara komponen manajer *client* dan mesin *server* pusat sebagai berikut:

3.3.3.1 Registrasi Identitas Pada Client

Proses diawali pada sisi *client* dengan memindai seluruh direktori data mentah mahasiswa pada penyimpanan internal untuk memisahkan format penamaan folder menjadi komponen nomor induk mahasiswa (NRP) dan nama subjek secara terpisah. Setiap kali entitas identitas baru ditemukan, manajer *client* secara otomatis mengemas parameter data tersebut ke dalam paket JSON dan mengirimkannya ke *server* pusat melalui protokol REST API HTTP POST, sekaligus mendaftarkan salinan data tersebut ke dalam pangkalan data SQLite lokal perangkat. Implementasi fungsi penemuan dan pengiriman identitas awal pada sisi *client* ini ditunjukkan pada Kode Sumber 3.2.

Listing 3.2: Registrasi Identitas Pada *Client*.

```
1 import os
2
3 def run_discovery_phase(self):
4     try:
5         # Tentukan folder mahasiswa
6         path = os.path.join(self.raw_data_path, "students")
7         if not os.path.exists(path):
8             path = self.raw_data_path
9
10        # Membaca daftar sub-direktori mahasiswa
11        folders = [
12            f for f in os.listdir(path)
13            if os.path.isdir(os.path.join(path, f))
14        ]
15        for folder in sorted(folders):
16            # Memisah nama folder menjadi NRP dan Nama
17            nrp = folder.split('_')[0] if "_" in folder else folder
18            name = folder.split('_')[1] if "_" in folder else nrp
19
20            # Kirim identitas ke server pusat menggunakan variabel
21            url = f"{self.server_api_url}{api_training_get_label}"
22            payload = {
23                "nrp": nrp,
24                "name": name,
25                "client_id": self.client_id
26            }
```

```

27         self._safe_request("POST", url, json=payload)
28
29         # Daftarkan ke SQLite lokal perangkat
30         db = SessionLocal()
31         try:
32             user = db.query(UserLocal).filter_by(
33                 user_id=nrp
34             ).first()
35             if not user:
36                 db.add(UserLocal(user_id=nrp, name=name))
37                 db.commit()
38         finally:
39             db.close()
40
41         # Beritahu server bahwa discovery selesai
42         done_url = (
43             f"{self.server_api_url}"
44             f"{api_clients_discovery_done}"
45         )
46         done_payload = {"client_id": self.client_id}
47         self._safe_request("POST", done_url, json=done_payload)
48     except Exception:
49         pass

```

3.3.3.2 Penanganan Registrasi Pada Server

Pada sisi penerima, *server* menangkap kiriman paket registrasi dari seluruh terminal *client* yang aktif melalui fungsi penerimaan *endpoint* khusus. *Server* memeriksa keberadaan data nomor induk mahasiswa di dalam *database* pusat. Jika data tersebut belum terdaftar, entri pengguna baru akan dibuat secara otomatis dengan pengenal unik (*integer* ID) menggunakan mekanisme *auto-increment* pada pangkalan data PostgreSQL. Pendekatan terpusat ini memastikan setiap subjek memiliki satu representasi nomor label kelas yang mutlak dan terhindar dari risiko tabrakan identitas lintas perangkat *edge*. Alur penanganan registrasi dan penjaminan ID unik pada *server* ditunjukkan pada Kode Sumber 3.3.

Listing 3.3: Penanganan Registrasi Pada *Server*.

```

1 import base64
2
3 @app.post(api_training_get_label)
4 async def get_label(data: dict, db: Session = Depends(get_db)):
5     nrp = data.get("nrp")
6     name = data.get("name", "Unknown")
7     edge_id = data.get("client_id", "edge-1")
8     emb_b64 = data.get("embedding")
9
10    # Dekode embedding dari Base64
11    emb_bytes = (
12        base64.b64decode(emb_b64)
13        if emb_b64 else None
14    )
15
16    # Periksa apakah NRP sudah terdaftar
17    user = db.query(UserGlobal).filter_by(nrp=nrp).first()
18    if not user:
19        # Tambah entri baru (ID terbuat secara otomatis)

```

```

20         user = UserGlobal(
21             nrp=nrp,
22             name=name,
23             registered_edge_id=edge_id,
24             embedding=emb_bytes
25         )
26         db.add(user)
27         db.commit()
28         db.refresh(user)
29     else:
30         # Perbarui nama atau embedding
31         user.name = name
32         if emb_bytes:
33             user.embedding = emb_bytes
34         db.commit()
35
36     # Masukkan NRP ke daftar data aktif
37     if nrp not in fl_manager.received_data:
38         fl_manager.received_data.append(nrp)
39
40     return {"nrp": nrp, "label": user.user_id}

```

3.3.3.3 Pembentukan Global Label Map Server

Setelah seluruh data dari *node client* terkumpul secara kolektif, *server* mengeksekusi fungsi penarikan data terpusat untuk membentuk struktur kluster matriks yang seragam. Sistem menarik seluruh baris identitas pengguna global dari *database*, mengurutkannya secara teratur berpasangan pada urutan string karakter nomor induk mahasiswa, lalu mengemas barisan data tersebut ke dalam sebuah *array* indeks terpadu. Larik data urutan yang terstandar inilah yang menjadi struktur acuan tunggal dimensi lapisan luaran (*output layer classifier*) pada arsitektur model. Logika pembentukan peta label terpadu di sisi *server* ditunjukkan pada Kode Sumber 3.4.

Listing 3.4: Pembentukan Global Label Map Server.

```

1 @app.get(api_training_label_map)
2 async def get_label_map(db: Session = Depends(get_db)):
3     # Ambil semua identitas terdaftar, urutkan berdasarkan NRP
4     users = db.query(UserGlobal).order_by(UserGlobal.nrp).all()
5     # Kembalikan array NRP terurut
6     return [u.nrp for u in users]

```

3.3.3.4 Sinkronisasi Global Label Map Client

Pada fase akhir sebelum proses pra-pemrosesan citra dijalankan, komponen manajer *client* memicu fungsi penarikan balik peta klasifikasi global. *Client* mengunduh *array* indeks pemetaan terpadu dari *server* melalui protokol HTTP GET, menghitung total dimensi kelas baru untuk melakukan ekspansi ukuran matriks bobot pada *classifier head* model lokal secara otomatis, lalu membekukan hasilnya ke dalam berkas konfigurasi JSON lokal. Berkas konfigurasi objek inilah yang menjadi acuan utama sistem *client* dalam memetakan target folder data latih secara seragam di seluruh ekosistem *Federated Learning*. Logika pengodean sinkronisasi umpan balik pada *client* ditunjukkan pada Kode Sumber 3.5.

Listing 3.5: Sinkronisasi Global Label Map Client.

```

1 import os

```

```

2 import json
3
4 def sync_label_map(self):
5     try:
6         self._ensure_models_loaded()
7
8         # Request daftar label map dari server menggunakan variabel
9         url = f"{self.server_api_url}{api_training_label_map}"
10        res = self._safe_request("GET", url)
11
12        if res and res.status_code == 200:
13            self.client.label_map = res.json()
14            self.num_classes = len(self.client.label_map)
15
16            # Perluas classifier head jika kelas bertambah
17            if self.head is not None:
18                curr_classes = self.head.weight.shape[0]
19                if curr_classes != self.num_classes:
20                    new_map = {
21                        nrp: idx for idx, nrp in
22                        enumerate(self.client.label_map)
23                    }
24                    self.head = (
25                        self.client.trainer
26                        .update_head(self.num_classes, new_map)
27                    )
28
29            # Simpan peta label ke file JSON lokal
30            path = os.path.join(
31                self.data_path, "models", "label_map.json"
32            )
33            os.makedirs(os.path.dirname(path), exist_ok=True)
34            with open(path, "w") as f:
35                json.dump(self.client.label_map, f)
36            return True
37        except Exception:
38            pass
39        return False

```

3.3.4 Image Preprocessing

Tahap *image preprocessing* merupakan fase operasional terisolasi pada sisi *client* yang bertujuan untuk membersihkan, menyeimbangkan geometri posisi spasial, serta menstandarkan dimensi matriks piksel wajah mahasiswa sebelum diekstraksi menjadi vektor karakteristik. Langkah ini dirancang untuk mengondisikan data visual mentah agar memenuhi kriteria format masukan arsitektur jala saraf MobileFaceNet secara seragam. Seluruh siklus manipulasi data statis ini dijalankan menggunakan unit CPU pada perangkat keras *client* demi menjaga efisiensi konsumsi daya melalui mekanisme pemuatan model detektor secara tertunda (*lazy loading*). Mekanisme ini dibagi ke dalam empat tahapan proses operasional sebagai berikut:

3.3.4.1 Deteksi dan Alignment Wajah

Proses diawali dengan melokalisasi kotak pembatas wajah memanfaatkan arsitektur MTCNN sekaligus mengalkulasi matriks transformasi afinitas dua dimensi berdasarkan koordinat lima titik marka penanda biometrik meliputi mata, hidung, dan mulut. Citra wajah kemudian diro-

tasi secara matematis berpatokan pada koordinat acuan standar agar posisi kemiringan kepala mahasiswa tegak simetris, lalu dipotong secara presisi ke dalam dimensi potret wajib 96x112 piksel. Jika proses transformasi afinitas mengalami kegagalan, sistem menyediakan fungsi jala pengaman (*fallback*) untuk memotong gambar secara statis mengikuti koordinat kotak pembatas awal dengan margin tambahan. Implementasi pengodean untuk proses alih geometri spasial ini ditunjukkan pada Kode Sumber 3.6.

Listing 3.6: Deteksi dan *Alignment* Wajah.

```

1 import cv2
2 import numpy as np
3 from PIL import Image
4
5 def detect_face(self, img, save_path=None):
6     try:
7         orig_w, orig_h = img.size
8         max_size = 640
9         # Downscaling gambar agar proses MTCNN lebih cepat
10        if orig_w > max_size or orig_h > max_size:
11            scale = max_size / max(orig_w, orig_h)
12            new_size = (
13                int(orig_w * scale),
14                int(orig_h * scale)
15            )
16            img_detect = img.resize(new_size, Image.BILINEAR)
17        else:
18            img_detect = img
19            scale = 1.0
20
21        # Inferensi deteksi wajah menggunakan MTCNN
22        boxes, probs, landmarks = self.mtcnn.detect(
23            img_detect, landmarks=True
24        )
25        if boxes is None or len(boxes) == 0:
26            return None, None, 0.0
27
28        if scale != 1.0:
29            boxes = boxes / scale
30            if landmarks is not None:
31                landmarks = landmarks / scale
32
33        face_img = None
34
35        # Affine Alignment berbasis koordinat 5 titik landmark
36        if landmarks is not None and landmarks[0] is not None:
37            try:
38                src = np.array(landmarks[0], dtype=np.float32)
39                M, _ = cv2.estimateAffinePartial2D(
40                    src, CANONICAL_LANDMARKS, method=cv2.LMEDS
41                )
42                if M is not None:
43                    img_cv = cv2.cvtColor(
44                        np.array(img), cv2.COLOR_RGB2BGR
45                    )
46                    aligned = cv2.warpAffine(
47                        img_cv, M, (96, 112),
48                        flags=cv2.INTER_LINEAR,

```



```

49         borderMode=cv2.BORDER_REPLICATE
50     )
51     face_img = Image.fromarray(
52         cv2.cvtColor(aligned, cv2.COLOR_BGR2RGB)
53     )
54     except Exception:
55         pass
56
57     # Fallback Bounding Box Crop jika alignment gagal
58     if face_img is None:
59         box = boxes[0]
60         margin = 20
61         x1 = max(0, int(box[0] - margin / 2))
62         y1 = max(0, int(box[1] - margin / 2))
63         x2 = min(img.width, int(box[2] + margin / 2))
64         y2 = min(img.height, int(box[3] + margin / 2))
65
66         face_img = img.crop((x1, y1, x2, y2)).resize(
67             (96, 112), Image.BILINEAR
68         )
69
70     if save_path and face_img:
71         face_img.save(save_path)
72
73     return face_img, boxes[0], probs[0]
74 except Exception:
75     return None, None, 0.0

```

3.3.4.2 Persiapan Model Tensor

Proses kedua berfungsi untuk mengonversi format data citra hasil pemotongan wajah menjadi bentuk objek matriks bilangan desimal multidimensi (Tensor) PyTorch. Melalui fungsi penyiapan ini, nilai intensitas piksel dipetakan ulang lewat fungsi transformasi normalisasi dengan rentang nilai parameter rata-rata (*mean*) dan deviasi standar (*standard deviation*) yang disesuaikan pada angka 0,5 guna menstabilkan gradien komputasi. Sistem kemudian menyisipkan dimensi *batch* tambahan pada sumbu pertama matriks agar struktur data tensor siap diumpankan secara langsung menuju lapisan masukan model. Logika transformasi penyiapan data masukan ini ditunjukkan pada Kode Sumber 3.7.

Listing 3.7: Transformasi dan Normalisasi Nilai Tensor L2.

```

1 import torch
2 import torchvision.transforms.functional as TF
3 from PIL import Image
4
5 def prepare_for_model(self, face_data):
6     if face_data is None:
7         return None
8
9     # Konversi ke format PIL Image
10    if isinstance(face_data, torch.Tensor):
11        if face_data.max() > 2.0:
12            face_data = face_data / 255.0
13            face_img = TF.to_pil_image(face_data.clamp(0, 1))
14        else:
15            face_img = face_data

```

```

16
17     # Resize ke dimensi masukan MobileFaceNet (96x112)
18     if face_img.size != (96, 112):
19         face_img = face_img.resize((96, 112), Image.BILINEAR)
20
21     face_tensor = TF.to_tensor(face_img)
22     normalized_tensor = self.normalize(face_tensor)
23
24     # Tambah dimensi batch (1, C, H, W)
25     return normalized_tensor.unsqueeze(0).to(DEVICE)

```

3.3.4.3 Perhitungan Tingkat Kekaburan Gambar

Proses ketiga bertindak sebagai unit penilai kualitas fisik gambar untuk mendeteksi tingkat ketajaman fokus lensa kamera terhadap objek wajah mahasiswa. Langkah evaluasi dilakukan dengan mengubah ruang warna berkas gambar dari format warna asal menjadi skala keabuan (*grayscale*) terisolasi. Sistem selanjutnya menerapkan operator *kernel* matematika Laplacian untuk menghitung nilai varians turunan kedua dari matriks gradien, di mana skor yang dihasilkan merepresentasikan tingkat ketajaman tepi objek visual secara kuantitatif. Logika perhitungan skor keaburan citra ini ditunjukkan pada Kode Sumber 3.8.

Listing 3.8: Perhitungan Skor Ketajaman Pikel Laplacian.

```

1 import cv2
2
3 def get_blur_score(self, image_path):
4     try:
5         img = cv2.imread(image_path)
6         if img is None:
7             return 0
8         # Konversi gambar ke skala abu-abu
9         gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
10        # Variansi Laplacian untuk skor ketajaman
11        return cv2.Laplacian(gray, cv2.CV_64F).var()
12    except Exception:
13        return 0

```

3.3.4.4 Seleksi Kualitas Citra Terbaik Berbasis Ranking

Proses keempat mengintegrasikan fungsi seleksi massal untuk menyaring dan memilih sejumlah N berkas foto wajah terbaik dari direktori penyimpanan sementara. Fungsi ini mendata seluruh file citra, mengurutkan daftar gambar berdasarkan pemetaan skor *Laplacian Variance* dari nilai tertinggi ke terendah, lalu mengekstrak sampel foto tertajam sesuai dengan jumlah batasan yang ditentukan. Guna mempercepat durasi eksekusi data, skrip memanfaatkan sistem pencatatan berkas riwayat berbasis objek dokumen konfigurasi agar kalkulasi matematika tidak perlu diproses ulang dari awal. Logika pengurutan kualitas citra terbaik ini ditunjukkan pada Kode Sumber 3.9.

Listing 3.9: Seleksi Kualitas Citra Berbasis Ranking Ketajaman.

```

1 import os
2 import json
3
4 def select_best_faces(self, folder_path, n=50):
5     if not os.path.exists(folder_path):

```

```

6         return []
7
8     final_cache_path = os.path.join(
9         folder_path, ".selection_cache.json"
10    )
11    scores_cache_path = os.path.join(
12        folder_path, ".laplacian_scores.json"
13    )
14
15    # Gunakan cache hasil seleksi sebelumnya jika ada
16    if os.path.exists(final_cache_path):
17        try:
18            with open(final_cache_path, "r") as f:
19                cache_data = json.load(f)
20                if cache_data.get("n") == n:
21                    return cache_data.get("filenames", [])
22        except Exception:
23            pass
24
25    all_imgs = sorted([
26        f for f in os.listdir(folder_path)
27        if f.lower().endswith(('.jpg', '.jpeg', '.png'))
28    ])
29
30    if not all_imgs:
31        return []
32
33    scored_map = {}
34    if os.path.exists(scores_cache_path):
35        try:
36            with open(scores_cache_path, "r") as f:
37                scored_map = json.load(f)
38        except Exception:
39            pass
40
41    needs_save = False
42    for img_name in all_imgs:
43        if img_name in scored_map:
44            continue
45
46        img_path = os.path.join(folder_path, img_name)
47        scored_map[img_name] = self.get_blur_score(img_path)
48        needs_save = True
49
50    if needs_save:
51        try:
52            with open(scores_cache_path, "w") as f:
53                json.dump(scored_map, f)
54        except Exception:
55            pass
56
57    # Urutkan dan pilih N foto tertajam
58    scored_list = [
59        (name, score) for name, score in scored_map.items()
60    ]
61    scored_list.sort(key=lambda x: x[1], reverse=True)
62    selected = [s[0] for s in scored_list[:n]]

```

```

63
64     # Simpan hasil seleksi ke cache
65     try:
66         with open(final_cache_path, "w") as f:
67             json.dump({"n": n, "filenames": selected}, f)
68     except Exception:
69         pass
70
71     return selected

```

3.3.5 Enkripsi Embedding

Tahap enkripsi kriptografi merupakan fase pengamanan data biometrik pada sisi *client* yang bertujuan untuk melindungi kerahasiaan vektor karakteristik (*face embedding*) mahasiswa sebelum disimpan ke dalam pangkalan data lokal maupun ditransmisikan di dalam jaringan. Langkah ini dirancang sebagai jaring pengaman privasi untuk memastikan bahwa jika pangkalan data lokal pada perangkat *edge* berhasil ditembus oleh pihak luar, data vektor wajah tersebut tetap tidak dapat disalahgunakan atau direkonstruksi kembali menjadi citra wajah asli tanpa kunci enkripsi yang sah. Proses pengamanan ini diimplementasikan menggunakan algoritma standar industri AES-256 bit dalam mode operasi *Cipher Block Chaining* (CBC). Fungsi operasional yang bertanggung jawab atas siklus pengamanan data biometrik ini ditunjukkan pada Kode Sumber 3.10.

Listing 3.10: Enkripsi Menggunakan AES-256.

```

1 import os
2 import pickle
3 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, ←
   modes
4 from cryptography.hazmat.primitives import padding
5 from cryptography.hazmat.backends import default_backend
6
7 def encrypt_embedding(self, embedding_numpy):
8     # Serialisasi array numpy embedding wajah menjadi bytes
9     data_bytes = pickle.dumps(embedding_numpy)
10
11     # Buat Initialization Vector (IV) acak sepanjang 16 bytes
12     iv = os.urandom(16)
13
14     # Inisialisasi Cipher AES-256 dalam mode CBC
15     cipher = Cipher(
16         algorithms.AES(self.key),
17         modes.CBC(iv),
18         backend=default_backend()
19     )
20     encryptor = cipher.encryptor()
21
22     # Lakukan padding data bytes menggunakan standar PKCS7
23     padder = padding.PKCS7(128).padder()
24     padded_data = (
25         padder.update(data_bytes) +
26         padder.finalize()
27     )
28
29     # Proses enkripsi data hasil padding
30     encrypted_data = (

```

```

31         encryptor.update(padded_data) +
32         encryptor.finalize()
33     )
34
35     return encrypted_data, iv

```

3.3.6 Pelatihan Model

Tahap pelatihan model merupakan fase pembelajaran utama dalam sistem absensi terdistribusi di mana bobot model dioptimasi secara kolaboratif menggunakan pendekatan *Federated Learning* (*pFedFace*). Fase ini membagi proses pembaruan parameter menjadi beberapa tahap utama: pembekuan parsial, augmentasi citra secara dinamis, proses *training* lokal dengan regularisasi *FedProx*, dan agregasi parameter terpusat di *server*. Alur ini memastikan model *edge* dapat belajar dengan konsumsi RAM yang rendah dan tetap patuh pada kerangka privasi data.

3.3.6.1 Pembekuan Parsial (Layer Freezing)

Sebelum siklus pelatihan dimulai pada perangkat *edge*, sistem membekukan lapisan-lapisan awal *backbone MobileFaceNet*. Pembekuan lapisan awal (*early freezing*) ini menonaktifkan kalkulasi gradien pada lapisan ekstraksi fitur dasar, sehingga menghemat konsumsi memori komputasi perangkat *edge*. Pengondisian pembekuan lapisan ini ditunjukkan pada Kode Sumber 3.11.

Listing 3.11: Pengaturan Pembekuan Lapisan Model.

```

1 def set_model_freeze(model, freeze_mode="early"):
2     for param in model.parameters():
3         param.requires_grad = True
4
5     if freeze_mode == "none":
6         return
7
8     # Bekukan lapisan input awal MobileFaceNet
9     if freeze_mode == "early":
10        for param in model.conv1.parameters():
11            param.requires_grad = False
12        for param in model.dw_conv1.parameters():
13            param.requires_grad = False
14
15        # Bekukan 12 blok bottleneck awal pada backbone
16        for i in range(12):
17            for param in model.blocks[i].parameters():
18                param.requires_grad = False
19
20        # Bekukan seluruh parameter backbone
21    elif freeze_mode == "backbone":
22        for param in model.parameters():
23            param.requires_grad = False

```

3.3.6.2 Augmentasi Data On-the-Fly

Selama proses pemuatan data oleh *DataLoader*, gambar wajah dimodifikasi secara dinamis di RAM menggunakan transformasi acak. Langkah ini menghasilkan variasi spasial dan visual baru di setiap *epoch* tanpa mengubah berkas citra asli pada penyimpanan fisik. Logika augmentasi data dan pemuatan dinamis ditunjukkan pada Kode Sumber 3.12.

Listing 3.12: Augmentasi dan Pemuatan Data Latih.

```
1 import torch
2 from PIL import Image
3 from torchvision import transforms
4 from torchvision.transforms import InterpolationMode
5
6 # Konfigurasi pipa augmentasi citra
7 self.transform = transforms.Compose([
8     transforms.Resize(
9         (112, 96),
10        interpolation=InterpolationMode.BILINEAR
11    ),
12    transforms.RandomHorizontalFlip(),
13    transforms.RandomRotation(degrees=20),
14    transforms.RandomPerspective(distortion_scale=0.2, p=0.3),
15    transforms.ColorJitter(
16        brightness=0.5, contrast=0.5,
17        saturation=0.4, hue=0.1
18    ),
19    transforms.RandomGrayscale(p=0.2),
20    transforms.RandomAutocontrast(p=0.2),
21    transforms.RandomApply([
22        transforms.GaussianBlur(
23            kernel_size=(3, 3), sigma=(0.1, 2.0)
24        )
25    ], p=0.4),
26    transforms.RandomAdjustSharpness(
27        sharpness_factor=2, p=0.2
28    ),
29    transforms.ToTensor(),
30    transforms.Normalize(
31        mean=[0.5, 0.5, 0.5],
32        std=[0.50196, 0.50196, 0.50196]
33    ),
34    transforms.RandomErasing(p=0.1)
35 ])
36
37 # Pemuatan sampel data latih secara dinamis
38 def __getitem__(self, idx):
39     sample = self.samples[idx]
40     label = sample['label']
41
42     # Baca gambar jika tipe data berupa file citra
43     if sample['type'] == "image":
44         image = Image.open(sample['path']).convert('RGB')
45         if self.transform:
46             image = self.transform(image)
47         return image, label, False
48     else:
49         # Kembalikan data embedding memori global
50         return sample['data'], label, True
```

3.3.6.3 Pelatihan Lokal

Pelatihan model lokal dijalankan pada *client* menggunakan optimasi *SGD* dan *ArcFace Margin Loss*. Guna mengatasi masalah ketidakseimbangan distribusi data *non-IID*, ditam-

bahkan penalti regulasi jarak L2 (*proximity loss*) antara parameter model lokal dengan model global acuan. Logika utama pelatihan lokal *client* ditunjukkan pada Kode Sumber 3.13.

Listing 3.13: Loop Pelatihan Lokal *Client*.

```
1 import copy
2 import torch
3 import torch.nn as nn
4 from torch.utils.data import DataLoader
5
6 def train(
7     self, epochs, lr, round_num,
8     global_embeddings=None, label_map=None, mu=0.05
9 ):
10     # Inisialisasi dataset dan loader
11     dataset = FaceDataset(
12         self.data_path,
13         global_embeddings=global_embeddings,
14         transform=self.transform
15     )
16     dataloader = DataLoader(
17         dataset, batch_size=32,
18         shuffle=True, collate_fn=hybrid_collate
19     )
20
21     # Bekukan lapisan awal model lokal
22     set_model_freeze(self.backbone, freeze_mode="early")
23     global_ref = copy.deepcopy(self.backbone.state_dict())
24
25     # Pengaturan optimizer SGD dan kriteria loss
26     optimizer = torch.optim.SGD(
27         self.backbone.parameters(),
28         lr=lr, momentum=0.9, nesterov=True
29     )
30     criterion = nn.CrossEntropyLoss()
31
32     for epoch in range(epochs):
33         self.backbone.train()
34         self.head.train()
35
36         for imgs, embs, labels, is_embedding in dataloader:
37             imgs = imgs.to(self.device)
38             embs = embs.to(self.device)
39             labels = labels.to(self.device)
40             optimizer.zero_grad()
41
42             features_local = torch.zeros(
43                 (labels.size(0), 128), device=self.device
44             )
45             img_mask, emb_mask = ~is_embedding, is_embedding
46
47             # Forward pass citra wajah ke backbone
48             if img_mask.any():
49                 features_local[img_mask] = (
50                     self.backbone(imgs[img_mask])
51                 )
52             if emb_mask.any():
53                 features_local[emb_mask] = embs[emb_mask]
```

```

54
55     # Hitung loss klasifikasi ArcFace
56     outputs = self.head(features_local, labels)
57     ce_loss = criterion(outputs, labels)
58
59     # Hitung penalti regulasi jarak L2 (FedProx)
60     prox_loss = torch.tensor(0.0, device=self.device)
61     for name, param in self.backbone.named_parameters():
62         if name in global_ref:
63             diff_norm = (
64                 torch.norm(param - global_ref[name])**2
65             )
66             prox_loss += (mu / 2) * diff_norm
67
68     # Total Loss = ArcFace Loss + Proximity Loss
69     loss = ce_loss + prox_loss
70
71     # Backward pass & update bobot
72     loss.backward()
73     optimizer.step()

```

3.3.6.4 Agregasi Parameter pada Server (FedAvg)

Server mengagregasikan parameter *backbone* dari seluruh *client* aktif menggunakan pembobotan rata-rata berbasis *Flower*. Guna mempertahankan adaptasi personalisasi (*personalization*) tingkat *client*, statistik lapisan *Batch Normalization* (BN) tidak disertakan dalam proses agregasi global (*global aggregation*). Logika agregasi parameter konvolusi *backbone* ditunjukkan pada Kode Sumber 3.14.

Listing 3.14: Agregasi Model Backbone Global.

```

1 import torch
2 import flwr as fl
3
4 def aggregate_fit(
5     self, server_round: int, results: list, failures: list
6 ):
7     # Agregasi parameter terbobot Flower
8     aggregated_parameters, aggregated_metrics = (
9         super().aggregate_fit(
10             server_round, results, failures
11         )
12     )
13     if aggregated_parameters is None:
14         return None, aggregated_metrics
15
16     # Urutkan bobot hasil agregasi ke NumPy array
17     params_np = (
18         fl.common.parameters_to_ndarrays(
19             aggregated_parameters
20         )
21     )
22     sd = self.load_previous_global_weights()
23     all_keys = list(sd.keys())
24
25     # Filter lapisan konvolusi (tanpa BN) untuk pFedFace
26     bn_patterns = ['bn', 'running_', 'num_batches_tracked']

```

```

27     conv_keys = [
28         k for k in all_keys
29         if not any(x in k.lower() for x in bn_patterns)
30     ]
31
32     # Tentukan kunci target parameter yang akan diperbarui
33     target_keys = (
34         conv_keys if len(params_np) == len(conv_keys)
35         else all_keys
36     )
37
38     # Rekonstruksi parameter ke Tensor PyTorch
39     backbone_params = {}
40     bn_params = {}
41     for i, k in enumerate(target_keys):
42         if i < len(params_np):
43             val = torch.from_numpy(params_np[i].copy())
44             if any(x in k.lower() for x in bn_patterns):
45                 bn_params[k] = val
46             else:
47                 backbone_params[k] = val
48
49     # Perbarui bobot model global baru
50     sd.update(backbone_params)
51     if bn_params:
52         sd.update(bn_params)
53
54     # Simpan bobot teragregasi ke database
55     self.save_new_global_weights(sd)
56     return aggregated_parameters, aggregated_metrics

```

3.3.7 Evaluasi Sistem

Tahap evaluasi sistem dibagi menjadi dua fokus pengukuran utama: evaluasi performa model untuk mengukur keandalan klasifikasi wajah, dan evaluasi konsumsi sumber daya (ekonomi) untuk mengukur dampak penggunaan daya komputasi serta transfer data jaringan (*network bandwidth*) dalam skema pembelajaran terdistribusi.

3.3.7.1 Evaluasi Performa Model

Evaluasi performa model dilakukan baik di sisi *client* untuk memvalidasi model lokal, maupun di sisi *server* untuk mengukur performa agregasi model global (*global model aggregation*) secara menyeluruh:

A. Validasi Performa Lokal Client Evaluasi performa model lokal dijalankan pada akhir setiap ronde pelatihan *client* menggunakan *dataset* validasi lokal yang terpisah (*disjoint*). Pengukuran ini menerapkan teknik *Flip Trick* (mengkombinasikan *embedding* citra asli dan *mirror horizontal*) untuk mencerminkan akurasi sesungguhnya pada kondisi operasional. Implementasi *loop* evaluasi performa lokal ini ditunjukkan pada Kode Sumber 3.15.

Listing 3.15: Uji Coba Validasi Performa Model *Client*.

```

1 import torch
2 import torch.nn as nn
3 from torch.utils.data import DataLoader

```

```

4
5 def evaluate(self, global_embeddings=None, label_map=None):
6     # Memuat dataset validasi lokal
7     dataset = FaceDataset(
8         self.data_path,
9         global_embeddings=global_embeddings,
10        transform=self.val_transform,
11        mode="val", label_map=label_map
12    )
13    if len(dataset) < 2:
14        return 0.0, 0.0, len(dataset)
15
16    dataloader = DataLoader(
17        dataset, batch_size=16,
18        shuffle=False, collate_fn=hybrid_collate
19    )
20    criterion = nn.CrossEntropyLoss()
21    self.backbone.eval()
22    self.head.eval()
23
24    total_loss, correct, total = 0.0, 0, 0
25    with torch.no_grad():
26        for imgs, embs, labels, is_embedding in dataloader:
27            imgs = imgs.to(self.device)
28            embs = embs.to(self.device)
29            labels = labels.to(self.device)
30
31            features = torch.zeros(
32                (labels.size(0), 128), device=self.device
33            )
34            img_mask, emb_mask = ~is_embedding, is_embedding
35
36            if img_mask.any():
37                img_input = imgs[img_mask]
38
39                # Ekstraksi embedding asli & mirror (Flip Trick)
40                f_orig = self.backbone(img_input)
41                f_flip = self.backbone(torch.flip(img_input, [3]))
42
43                # Rata-rata dan normalisasi L2
44                features[img_mask] = (
45                    torch.nn.functional.normalize(
46                        f_orig + f_flip, p=2, dim=1
47                    )
48                )
49
50            if emb_mask.any():
51                features[emb_mask] = embs[emb_mask]
52
53            # Hitung logits kelas dan loss function
54            logits = self.head.get_logits(features)
55            outputs = self.head(features, labels)
56            loss = criterion(outputs, labels)
57
58            total_loss += loss.item()
59            _, predicted = torch.max(logits.data, 1)
60            total += labels.size(0)

```

```

61         correct += (predicted == labels).sum().item()
62
63     avg_loss = (
64         total_loss / len(dataloader)
65         if len(dataloader) > 0 else 0.0
66     )
67     accuracy = correct / total if total > 0 else 0.0
68     return avg_loss, accuracy, total

```

B. Agregasi Metrik Global Server Di sisi *server*, hasil evaluasi performa lokal dari seluruh *client* digabungkan menjadi metrik evaluasi global terpadu menggunakan metode rata-rata tertimbang (*weighted average*). *Server* mengagregasikan nilai *loss* dan akurasi (*accuracy*) baik dari data pelatihan lokal (*train dataset*) maupun data validasi lokal (*validation dataset*) berdasarkan jumlah sampel data masing-masing *client*. Implementasinya ditunjukkan pada Kode Sumber 3.16.

Listing 3.16: Fungsi Agregasi Metrik Performa *Server*.

```

1 def weighted_average(metrics: list) -> dict:
2     # Mengambil jumlah sampel per klien
3     examples = [m[0] for m in metrics]
4     total_examples = sum(examples)
5     if total_examples == 0:
6         return {}
7
8     # Rata-rata tertimbang metrik pelatihan & validasi
9     return {
10         "accuracy": sum([
11             m[1].get("accuracy", 0.0) * m[0] for m in metrics
12         ]) / total_examples,
13         "loss": sum([
14             m[1].get("loss", 0.0) * m[0] for m in metrics
15         ]) / total_examples,
16         "val_accuracy": sum([
17             m[1].get("val_accuracy", 0.0) * m[0] for m in metrics
18         ]) / total_examples,
19         "val_loss": sum([
20             m[1].get("val_loss", 0.0) * m[0] for m in metrics
21         ]) / total_examples,
22     }

```

3.3.7.2 Evaluasi Konsumsi Sumber Daya (Ekonomi)

Evaluasi ekonomi mencakup pengukuran konsumsi sumber daya sistem terdistribusi yang dibagi secara berimbang menjadi dua fokus pengukuran:

A. Pengukuran Sisi Client Perangkat *client* secara mandiri melacak durasi waktu pelatihan lokal (dalam detik) dan total konsumsi energi listrik CPU (dalam kWh) pada setiap putaran latihan lokal. Pengukuran durasi dilakukan dengan menghitung selisih waktu sistem operasi, sementara pengukuran energi listrik memanfaatkan pelacakan *offline* dari pustaka *CodeCarbon*. Penggabungan alur pelacakan durasi dan daya pada *client* ditunjukkan pada Kode Sumber 3.17.

Listing 3.17: Pelacakan Durasi dan Energi *Client*.

```
1 import time
2 from codecarbon import OfflineEmissionsTracker
3
4 # Inisialisasi awal waktu dan pencatat daya CodeCarbon
5 start_train_time = time.time()
6 tracker = OfflineEmissionsTracker(
7     country_iso_code="IDN",
8     measure_power_secs=15,
9     log_level="error",
10    save_to_file=False
11 )
12 tracker.start()
13
14 # Dapatkan total konsumsi kWh dan durasi detik pelatihan
15 energy_kwh = tracker.stop()
16 train_duration = time.time() - start_train_time
```

B. Pengukuran Sisi Server *Server* melacak tiga parameter utama secara terpadu: total durasi eksekusi ronde global, estimasi transfer data jaringan (*bandwidth* dalam MB) secara dinamis dari ukuran fisik berkas model di *disk*, serta total akumulasi konsumsi daya (*server* dan gabungan laporan energi *client*). Implementasi gabungan pelacakan sumber daya di *server* ditunjukkan pada Kode Sumber 3.18.

Listing 3.18: Pelacakan Durasi, *Bandwidth*, dan Akumulasi Energi *Server*.

```
1 import os
2 from datetime import datetime
3
4 # Hitung total durasi waktu per ronde global
5 round_duration = round(
6     datetime.now().timestamp() - round_start_time, 2
7 )
8
9 # Hitung transfer bandwidth nyata dari berkas model
10 bb_size = ECONOMICS["estimated_backbone_size_mb"]
11 if os.path.exists("data/backbone_pure.pth"):
12     bb_size = (
13         os.path.getsize("data/backbone_pure.pth")
14         / (1024 * 1024)
15     )
16 backbone_sync_mb = num_clients * bb_size * 2
17
18 # Akumulasi energi total server dan seluruh klien
19 if has_real_server_energy:
20     server_energy = max_server_energy
21 else:
22     server_energy = (
23         (round_duration / 3600)
24         * server_power_kw
25     )
26
27 client_energy_total = sum([
28     c.get("energy_kwh", 0) for c in clients_data
29 ])
```

```
30 total_energy = server_energy + client_energy_total
```

3.3.8 Proses Absensi

Tahap proses absensi merupakan fase operasional utama yang melibatkan interaksi dua arah antara terminal *client* secara *real-time* dan *server* pusat untuk melakukan konsolidasi data kehadiran mahasiswa.

3.3.8.1 Pipa Inferensi Real-Time Client

Proses inferensi dirancang agar berjalan efisien pada unit CPU perangkat *edge*. Skrip mengekstrak koordinat wajah menggunakan detektor MTCNN, menyelaraskan orientasi posisi geometris, melakukan ekstraksi fitur ganda (*Flip Trick*), dan menyeimbangkan *noise transient* dengan *buffer Temporal Voting* sepanjang rentang waktu 3 detik. Jika wajah berhasil dikenali, presensi dicatat ke *database* SQLite lokal sebelum disinkronkan ke *server*. Alur pengolahan inferensi presensi ini ditunjukkan pada Kode Sumber 3.19.

Listing 3.19: Alur Utama Inferensi Presensi Wajah *Client*.

```
1 import io
2 import os
3 import time
4 import base64
5 import torch
6 from PIL import Image
7
8 async def process_inference(self, image_b64: str, db: Session):
9     # Dekode citra base64 ke PIL Image
10    img_bytes = base64.b64decode(image_b64)
11    img_pil = Image.open(io.BytesIO(img_bytes)).convert('RGB')
12
13    # Deteksi dan crop wajah menggunakan MTCNN
14    face_tensor, box, prob = image_processor.detect_face(img_pil)
15    if face_tensor is None:
16        img_pil.close()
17        return {"matched": "Unknown", "confidence": 0}
18
19    threshold = self.fl_manager.inference_threshold
20    input_tensor = (
21        image_processor
22        .prepare_for_model(face_tensor)
23        .to(self.fl_manager.device)
24    )
25
26    # Ekstraksi Embedding Wajah dengan Flip Trick
27    with torch.no_grad():
28        self.fl_manager.backbone.eval()
29        emb_orig = self.fl_manager.backbone(input_tensor)
30        face_flipped = torch.flip(input_tensor, dims=[3])
31        emb_mirror = self.fl_manager.backbone(face_flipped)
32        query_emb = (
33            torch.nn.functional.normalize(
34                (emb_orig + emb_mirror) / 2, p=2, dim=1
35            )
36        )
37
```

```

38     # Logika buffer Temporal Voting (maksimal 3 detik)
39     now = time.time()
40     if now - self.fl_manager.last_face_time > 3.0:
41         self.fl_manager.prediction_buffer.clear()
42
43     self.fl_manager.prediction_buffer.append(query_emb)
44     self.fl_manager.last_face_time = now
45
46     # Rata-rata embedding temporal
47     mean_emb_tensor = (
48         torch.stack(
49             list(self.fl_manager.prediction_buffer)
50         ).mean(0)
51     )
52     mean_emb_tensor = (
53         torch.nn.functional.normalize(
54             mean_emb_tensor, p=2, dim=1
55         )
56     )
57     mean_emb = mean_emb_tensor.cpu().numpy()[0]
58
59     # Pencocokan identitas berbasis Cosine Similarity
60     local_refs = self._get_cached_identities(db)
61     best_match, confidence = (
62         identify_user_globally(
63             mean_emb, local_refs, threshold=threshold
64         )
65     )
66
67     # Validasi ambang batas kemiripan
68     user_id = best_match if confidence >= threshold else "Unknown"
69
70     # Simpan hasil absensi jika wajah dikenali
71     if user_id != "Unknown":
72         try:
73             new_attendance = AttendanceLocal(
74                 user_id=user_id,
75                 confidence=confidence,
76                 device_id=os.getenv("HOSTNAME", "terminal-1")
77             )
78             db.add(new_attendance)
79             db.commit()
80         except Exception:
81             db.rollback()
82
83     img_pil.close()
84     return {
85         "matched": user_id,
86         "confidence": float(confidence),
87         "box": box.tolist() if box is not None else None
88     }

```

3.3.8.2 Sinkronisasi Presensi Pusat (Server)

Di sisi *server*, terdapat sebuah *endpoint* API HTTP POST (`/api/attendance/sync`) untuk menerima unggahan *batch* log presensi dari *database* lokal *client* ketika koneksi jaringan

tersedia. *Server* mencocokkan NRP mahasiswa, memverifikasi integritas data untuk mencegah duplikasi, memvalidasi keberadaan perangkat *edge* di tabel registri, dan menyimpan log kehadiran global secara aman ke *database* PostgreSQL terpusat. Implementasi sinkronisasi absensi pada *server* ditunjukkan pada Kode Sumber 3.20.

Listing 3.20: API Sinkronisasi Kehadiran Global *Server*.

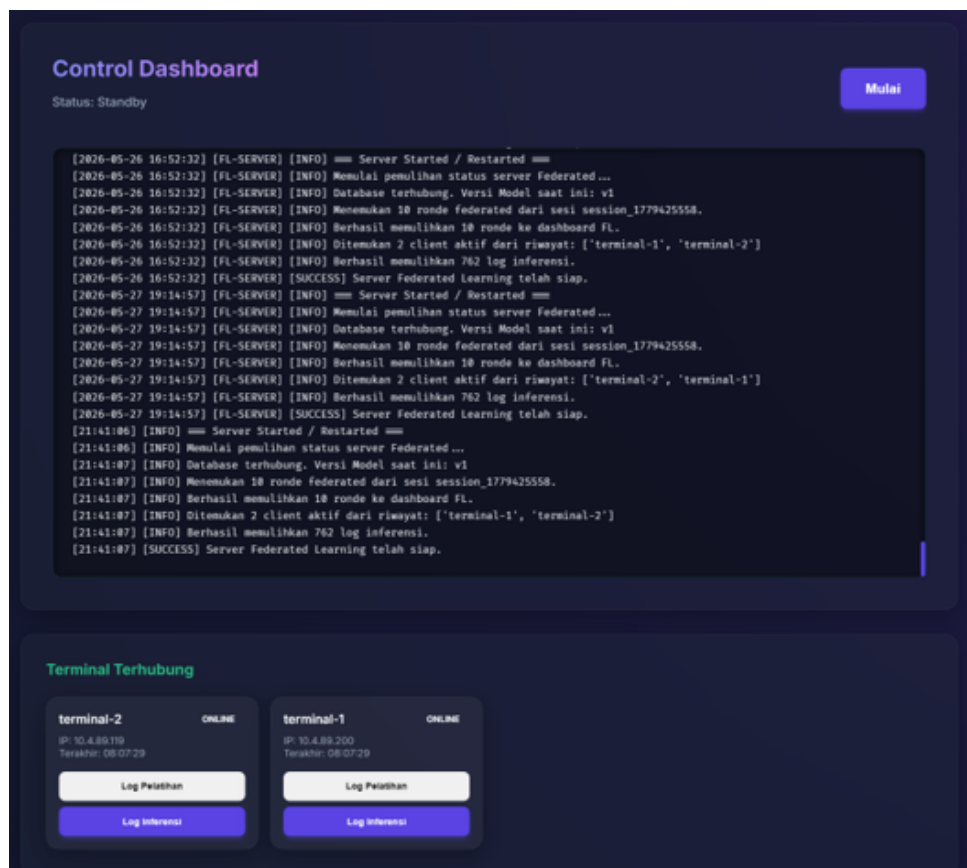
```
1 from typing import List, Dict, Any
2 from datetime import datetime
3 from fastapi import Body, Depends
4
5 @app.post("/api/attendance/sync")
6 async def sync_attendance(
7     records: List[Dict[str, Any]] = Body(...),
8     db: Session = Depends(get_db)
9 ):
10     new_records = 0
11     for rec in records:
12         try:
13             nrp = rec.get('user_id')
14             user = (
15                 db.query(UserGlobal)
16                 .filter_by(nrp=nrp).first()
17             )
18             if not user:
19                 continue
20
21             # Parsing timestamp ISO format
22             ts_str = rec['timestamp']
23             ts = datetime.fromisoformat(
24                 ts_str.replace('Z', '+00:00')
25             )
26
27             # Validasi duplikasi entry presensi
28             exists = (
29                 db.query(AttendanceRecap)
30                 .filter_by(user_id=user.user_id, timestamp=ts)
31                 .first()
32             )
33             if not exists:
34                 conf = rec.get('confidence', 0.0)
35                 client_id = rec.get('client_id', 'unknown-edge')
36
37                 # Simpan rekam absensi global mahasiswa
38                 new_item = AttendanceRecap(
39                     user_id=user.user_id,
40                     edge_id=client_id,
41                     timestamp=ts,
42                     confidence=conf
43                 )
44                 db.add(new_item)
45                 new_records += 1
46             except Exception:
47                 continue
48
49     db.commit()
50     return {"status": "success", "new_records": new_records}
```

3.3.9 Pengembangan Antarmuka Aplikasi

Tahap pengembangan antarmuka (*user interface*) bertujuan untuk menyediakan visualisasi interaktif bagi administrator dan pengguna dalam memantau jalannya sistem absensi terdistribusi. Pengembangan ini dibagi menjadi dua bagian utama, yaitu antarmuka pusat pada sisi *server* dan antarmuka operasional pada sisi terminal *client*.

3.3.9.1 Antarmuka Server

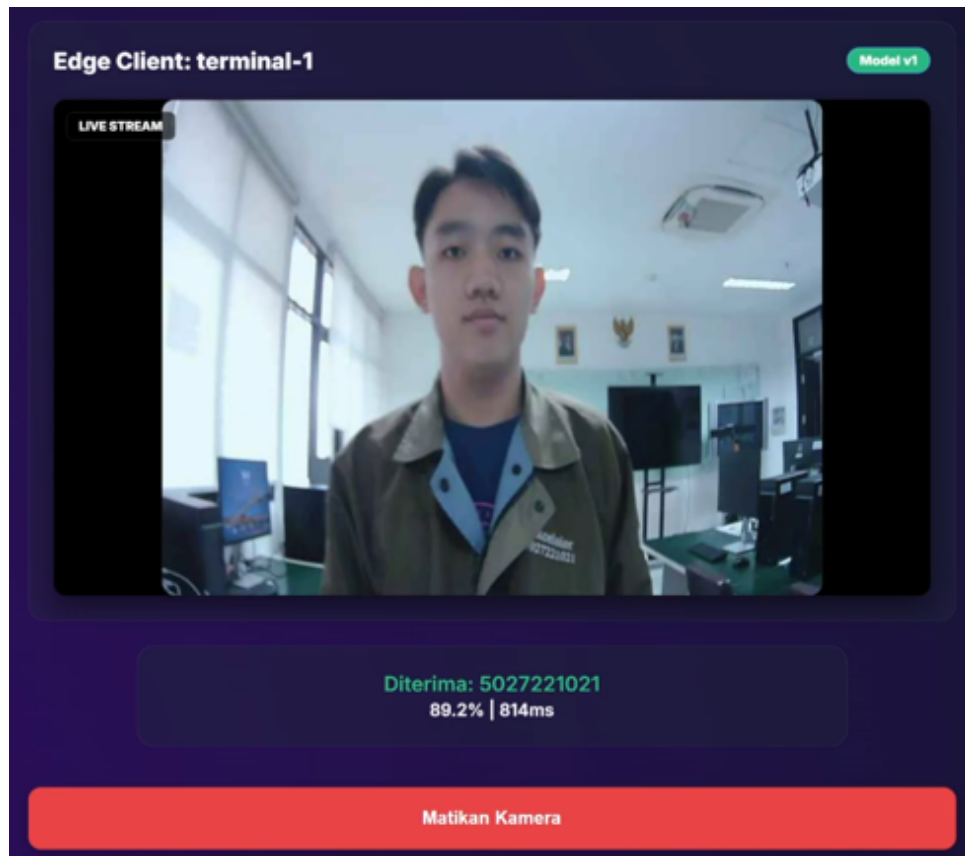
Antarmuka pada sisi *server* dirancang berbasis *web dashboard* terpusat yang memfasilitasi administrator untuk memantau status keaktifan masing-masing *client*, melihat visualisasi grafik metrik performa global (*accuracy* dan *loss*) dari setiap ronde pelatihan *Federated Learning*, serta mengelola rekapitulasi data kehadiran mahasiswa global yang disimpan di *database* pusat. Tampilan antarmuka dasbor utama ini dapat dilihat pada Gambar 3.1.



Gambar 3.1: Tampilan Antarmuka Dasbor Utama *Server*.

3.3.9.2 Antarmuka Client

Antarmuka pada sisi *client* dirancang untuk dioperasikan langsung di kelas menggunakan terminal *edge*. Halaman ini menyajikan tampilan *video stream* dari kamera secara *real-time*, mendeteksi wajah dengan kotak pembatas (*bounding box*), melakukan identifikasi menggunakan algoritma pencocokan *embedding* dan *temporal voting*, serta menampilkan notifikasi status kehadiran mahasiswa secara langsung sebelum datanya disinkronisasikan ke *server* pusat. Tampilan antarmuka terminal sisi *client* ini dapat dilihat pada Gambar 3.2.



Gambar 3.2: Tampilan Antarmuka Terminal Presensi Sisi *Client*.

DAFTAR PUSTAKA

- Anthropic. (2024). *Model context protocol (mcp) specification* [Referensi teknis untuk protokol]. <https://modelcontextprotocol.io/>
- Brecko, A., Kajati, E., Koziorek, J., & Zolotova, I. (2022). Federated learning for edge computing: A survey. *Applied Sciences (Switzerland)*, 12. <https://doi.org/10.3390/app12189124>
- Kim, J., Park, T., Kim, H., & Kim, S. (2021). Federated learning for face recognition. *Digest of Technical Papers - IEEE International Conference on Consumer Electronics, 2021-January*. <https://doi.org/10.1109/ICCE50685.2021.9427748>
- Nandi, A., Xhafa, F., & Kumar, R. (2023). A docker-based federated learning framework design and deployment for multi-modal data stream classification. *Computing*, 105, 2195–2229. <https://doi.org/10.1007/s00607-023-01179-5>
- Newton, I. (1687). Axioms or laws of motion. *Philosophiæ Naturalis Principia Mathematica*.
- Roket luar angkasa discovery. (2021). Retrieved February 3, 2021, from <https://airandspace.si.edu/explore-and-learn/topics/discovery/about.cfm>
- Srinu, N., Kumar, C. S. S., Senthil, M., & Babu, D. B. (2025). Smart attendance recording system utilizing cnn and edge computing techniques. *2025 5th International Conference on Soft Computing for Security Applications (ICSCSA)*, 551–558. <https://doi.org/10.1109/ICSCSA66339.2025.11170847>
- Staño, M., Hluchý, L., Krammer, P., & Hucko, M. (2025). Docker survey for flops efficiency. *Proceedings of the International Conference on Cybernetics and Informatics, K and I*. <https://doi.org/10.1109/KI64036.2025.10916451>
- Wang, Q., Zhu, Z., Shao, S., Alahi, A., Sadigh, D., & Wu, J. (2023). Voyager: An open-ended embodied agent with large language models. *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*. <https://arxiv.org/pdf/2305.16291.pdf>